

Consolidated Web Applications Portal & Architecture Guide

Nithya Mart Multi-Category E-Commerce & Service Ecosystem

Document Purpose & Abstract

This master documentation manual provides an exhaustive architectural, component, and operational breakdown of the **MERN Stack Team 4 Consolidated Workspace**. The platform consolidates 10 distinct React e-commerce and utility applications into a unified digital ecosystem under the central portal named **Nithya Mart**.

Built using React 19, Vite 8, Framer Motion 12, HTML5, and modern Vanilla CSS Design Tokens, this repository integrates luxury fashion, books, sweets, apparel rentals, fragrances, food court ordering, full-stack ice cream parlors, athletic sports gear, footwear, and event ticket booking into a cohesive multi-tenant environment.

Master Specification & Metadata Matrix

Specification Attribute	Project Implementation Details
Project Title	MERN Stack Team 4 Consolidated Application Portal (Nithya Mart)
Core Web Technologies	React 19.2.7, Vite 8.1.1, JavaScript ES6+, HTML5, Vanilla CSS3
UI & Motion Engine	Framer Motion 12.43, React Icons 5.7, 3D Perspective CSS Custom Properties
Utilities & HTTP Services	Axios 1.18, React QR Code 2.2, React Router DOM 7.18
Backend Engine (IceCreams)	Node.js, Express REST API, CORS Middleware, JSON API controllers
Repository Architecture	Multi-Application Git Subtree Monorepo with Sandboxed Iframe Viewports
Integrated Submodule Count	10 Self-Contained E-Commerce & Service Web Applications
Local Repository Root	`c:\Nithin_Academic\MERN_Stack_Team_4`

Document Structure Index (Pages 1–16)

- **Page 1:** Master Metadata & Document Purpose | **Page 2:** Executive Summary & Vision | **Page 3:** Subtree Architecture Strategy
- **Page 4:** Central Portal Hub (`src/App.jsx`) | **Page 5:** BAG Luxury Module | **Page 6:** BOOK Store Module
- **Page 7:** CHOCOLATES Boutique | **Page 8:** COSTUMES Rental Store | **Page 9:** FRAGRANCE Depot Module
- **Page 10:** BiteCourt FOOD Ordering | **Page 11:** Full-Stack ICECREAMS Parlour | **Page 12:** SPORTS Goods Store
- **Page 13:** SHOES Footwear Emporium | **Page 14:** TICKET BOOKING & Spiderman Showcase | **Page 15:** Technical Deep Dive
- **Page 16:** Developer Operations, Setup Guide & Strategic Roadmap

Executive Summary & Strategic Vision

1. Background & Academic Context

In modern web development, team projects often face the challenge of merging disparate software modules built by independent developers. Traditional merging techniques frequently result in broken dependencies, overwritten styling rules, and lost commit histories. To solve these engineering challenges, **MERN Stack Team 4** designed a consolidated web portal that aggregates 10 distinct, fully autonomous sub-applications into a single production workspace.

2. Strategic Platform Objectives

- **Centralized Multi-Tenant Aggregation:** Provide a unified entry point (Nithya Mart) that allows users to launch 10 independent e-commerce and utility platforms without page reloads.
- **Complete Autonomy for Submodules:** Ensure that each project folder maintains its own isolated `package.json`, asset directory, and routing logic without polluting sibling modules.
- **Preservation of Commit Histories:** Use Git Subtree strategies to merge developer branches, preserving full attribution and commit logs across all team members.
- **Modern Motion UI Design:** Deliver a state-of-the-art user interface featuring 3D hover perspective tilt, glassmorphism cards, dynamic dark mode context, and interactive canvas orbital mechanics.

3. Consolidated Submodule Portfolio

Module	Category	Core Business Domain & Operational Role
BAG	Shopping	Luxury Handbags, Office Briefcases, Travel Duffels & Accessories
BOOK	Education	Academic Textbooks, Bestseller Novels, Science & Genre Catalog
CHOCOLATES	Food & Sweets	Imported Confections, Cocoa Truffles, Custom Gift Box Creator
COSTUMES	Apparel	Theater Costumes, Party Wear & Halloween Rental Calculation System
FRAGRANCE	Beauty	Luxury Perfumes, Colognes, Scent Profiling & Aromatics Depot
Foodd	Food Court	BiteCourt Fast Food Ordering, Combos, Platters & Custom Hook Cart
IceCreams	Desserts	Full-Stack Express REST API + React Client Sundae Builder
SPORTS	Fitness	Athletic Gear, Sports Equipment, Jerseys & Specification Modals
Shoes	Apparel	Sneaker Emporium, Footwear Catalog & UK/US Size Matrix Selector
TICKETBOOKING	Entertainment	Cinema Seat Selector, QR Code Digital Pass & Spiderman Showcase

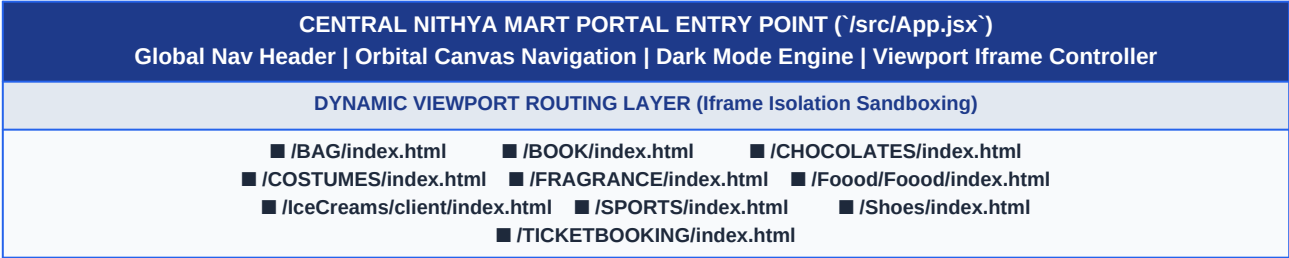
System Architecture & Git Subtree Mechanics

1. Monorepo vs Git Subtree Architectural Analysis

When designing multi-project repositories, developers choose between standard monorepos (Lerna/Nx), Git Submodules, and Git Subtrees. The table below details why **Git Subtrees** were selected for this workspace:

Integration Criteria	Git Submodules	Git Subtree Strategy (Chosen)
Cloning Complexity	High (Requires recursive init commands)	Zero (Standard `git clone` fetches all code)
Commit History	Pointers only; history stored remotely	Full Log Preserved directly in master branch
Directory Autonomy	High, but prone to pointer mismatch	Total Autonomy with folder-level isolation
Developer Overhead	Requires complex nested git commands	Seamless standard workflow for developers

2. Repository Architecture & Component Flow Diagram



3. Git Subtree Add Command Execution Reference

Each team module branch was integrated using subtree prefixes as shown below:

```
# Subtree Integration Command Pattern: # git subtree add --prefix= origin/ git subtree add --prefix=BAG
origin feature/bag-store git subtree add --prefix=BOOK origin feature/book-store git subtree add
--prefix=TICKETBOOKING origin feature/ticket-booking
```

Central Portal & Hub Engine (`src/App.jsx`)

1. Central Hub Functional Architecture

The file ``src/App.jsx`` serves as the central administrative cockpit of the workspace. Implemented with React 19, Framer Motion 12, and pure CSS custom properties, it manages global state, active project viewports, automated banner carousels, dark/light theme switching, and interactive orbital navigation maps.

2. Key Portal Sub-Systems & Technical Implementation

- **Interactive Orbital Universe Map:** A physics-inspired CSS module rendering a central core hub surrounded by concentric orbits (``orbit-track-1``, ``orbit-track-2``, ``orbit-track-3``) and orbiting module planets (``planet-bag``, ``planet-book``, ``planet-food``, ``planet-sports``). Hovering pauses planet rotation and triggers a glassmorphism info tooltip.
- **3D Perspective Card Tilt:** Dynamic mouse position calculation updating ``--x`` and ``--y`` CSS custom variables per card on hover, creating interactive 3D perspective transforms.
- **60 FPS Cursor Spotlight:** Real-time cursor position tracking updating ``--mouse-x`` and ``--mouse-y`` DOM root properties directly without triggering React virtual DOM re-renders.
- **Framer Motion Auto Carousel:** Animated hero showcase rotating featured collections (Luxury Bags, Chocolates, BiteCourt Food) with exit/enter transitions via ``AnimatePresence``.
- **Iframe Viewport Manager:** Sandboxed viewport rendering sub-application entry points (``/BAG/index.html``) with loading overlay spinners (``setIframeLoading``).

3. Portal Project Registry Data Schema

Module ID	Display Name	Target Path	Theme Class
<code>`bag`</code>	BAG E-Commerce	<code>`/BAG/index.html`</code>	<code>`theme-bag`</code>
<code>`book`</code>	BOOK Store	<code>`/BOOK/index.html`</code>	<code>`theme-book`</code>
<code>`chocolates`</code>	CHOCOLATES Shop	<code>`/CHOCOLATES/index.html`</code>	<code>`theme-chocolate`</code>
<code>`costumes`</code>	COSTUMES Store	<code>`/COSTUMES/index.html`</code>	<code>`theme-costume`</code>
<code>`fragrance`</code>	FRAGRANCE Depot	<code>`/FRAGRANCE/index.html`</code>	<code>`theme-fragrance`</code>
<code>`food`</code>	BiteCourt FOOD	<code>`/Food/Food/index.html`</code>	<code>`theme-food`</code>
<code>`icecream`</code>	ICECREAMS Parlour	<code>`/IceCreams/client/index.html`</code>	<code>`theme-icecream`</code>
<code>`sports`</code>	SPORTS Goods	<code>`/SPORTS/index.html`</code>	<code>`theme-sport`</code>
<code>`shoes`</code>	SHOES Emporium	<code>`/Shoes/index.html`</code>	<code>`theme-shoe`</code>
<code>`ticketbooking`</code>	TICKET BOOKING	<code>`/TICKETBOOKING/index.html`</code>	<code>`theme-ticket`</code>

4. Code Highlights — 3D Tilt & Mouse Spotlight Handler

```
// App.jsx - Real-time 60 FPS Cursor Tracking & 3D Tilt
useEffect(() => {
  const handleGlobalMouseMove = (e) => {
    document.documentElement.style.setProperty('--mouse-x', `${e.clientX}px`);
    document.documentElement.style.setProperty('--mouse-y', `${e.clientY}px`);
  };
  window.addEventListener('mousemove', handleGlobalMouseMove);
  return () => window.removeEventListener('mousemove', handleGlobalMouseMove);
}, []);
```

Submodule 1: BAG Luxury E-Commerce (`/BAG`)

1. Module Overview & Domain Scope

The **BAG E-Commerce Module** is a luxury handbag and accessories store module. It features high-resolution catalog displays, category filtration (Luxury Handbags, Professional Office Briefcases, Travel Duffel Bags, Leather Backpacks), price sliders, and persistent shopping cart management.

2. File Architecture & Directory Breakdown

File Path	Component Role & Description
`BAG/src/App.jsx`	Main router & layout wrapper integrating header navigation and page views
`BAG/src/context/CartContext.jsx`	React Context managing cart items array, item additions, quantity updates, and subtotal calculation
`BAG/src/pages/Home.jsx`	Landing hero page featuring luxury bag promos and category shortcuts
`BAG/src/pages/Products.jsx`	Product grid view with search query input, category pills, and price slider filter
`BAG/src/pages/ProductDetails.jsx`	Individual product detail view with image gallery, specs list, and Add to Cart action
`BAG/src/styles/App.css`	Custom styling rules, card glassmorphic hover animations, and responsive layout grids

3. Technical Features & UI Capabilities

- **Dynamic Price Slider:** Interactive range filter narrowing product view dynamically between \$50 and \$1,500.
- **Cart Badge Counter:** Real-time header badge reflecting total aggregated items in cart.
- **Responsive Asset Pipeline:** High-density image gallery under `src/assets/categories/` (`luxury-handbags.jpg`).

4. Code Highlights — Cart Context Provider Pattern (`CartContext.jsx`)

```
// BAG/src/context/CartContext.jsx - Cart Management Context export const CartProvider = ({ children }) => {
const [cart, setCart] = useState([]); const addToCart = (product) => { setCart((prev) => { const existing =
prev.find(item => item.id === product.id); if (existing) { return prev.map(item => item.id === product.id ?
{ ...item, qty: item.qty + 1 } : item); } return [...prev, { ...product, qty: 1 }]; }); }; const getSubtotal
= () => cart.reduce((sum, item) => sum + (item.price * item.qty), 0); return {children}; }
```

Submodule 2: BOOK Store Portal (`/BOOK`)

1. Module Purpose & Academic Scope

The **BOOK Store Module** is a modern online bookstore portal providing rapid catalog browsing, instant title/author search indexing, genre filtering (Fiction, Technology, Science, History, Fantasy), textbook recommendations, and order checkout simulation.

2. Component Hierarchy & File Structure

File Path	Component Role & Functionality
`BOOK/src/App.jsx`	Root component establishing Context providers, routing views, and main layout container
`BOOK/src/context/BookContext.jsx`	Global inventory context storing book items, active search query, and cart list
`BOOK/src/components/Navbar.jsx`	Navigation header with logo, genre links, search bar input, and cart drawer toggle button
`BOOK/src/components/BookCard.jsx`	Book card rendering cover thumbnail, rating stars, price, author, and Quick Add button
`BOOK/src/components/CartModal.jsx`	Slide-out modal displaying order itemization, subtotal, tax calculation, and Checkout trigger
`BOOK/src/App.css`	Grid layouts, typography styling, search highlighting, and cart transition animations

3. Technical Features & Search Indexing

- **Real-Time Search Indexer:** Filters book listings dynamically as the user types into the navbar search input, checking both title and author strings.
- **Genre Filter Pills:** Interactive category buttons allowing instant genre isolation.
- **Tax & Shipping Estimator:** Calculates 8% estimated sales tax and shipping cost automatically in the cart drawer.

4. Code Highlights — Real-Time Search Filtering Logic (`BookContext.jsx`)

```
// BOOK/src/context/BookContext.jsx - Search & Genre Filtering Engine
const getFilteredBooks = () => {
  return books.filter((book) => {
    const matchesSearch =
      book.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
      book.author.toLowerCase().includes(searchQuery.toLowerCase());
    const matchesGenre = activeGenre === 'All' ||
      book.genre === activeGenre;
    return matchesSearch && matchesGenre;
  });
};
```

Submodule 3: CHOCOLATES Boutique (`/CHOCOLATES`)

1. Module Overview & Product Domain

The **CHOCOLATES Shop Module** is an artisanal confection boutique platform highlighting imported Belgian truffles, dark cocoa slabs (70% - 90%), festive sweet assortments, and a custom **Gift Box Builder** that enables customers to assemble customized confection boxes.

2. File Architecture & Data Layer

File Path	Component Role & Description
`CHOCOLATES/src/App.jsx`	Main entry point configuring hero banner, product grid, and custom gift box modal
`CHOCOLATES/src/data/chocolateData.js`	Structured JSON dataset containing items, cocoa percentage, origin country, and pricing
`CHOCOLATES/src/components/ChocolateGrid.jsx`	Grid component displaying confectionery items with cocoa badge and Add to Box action
`CHOCOLATES/src/components/BoxBuilder.jsx`	Interactive custom gift box configurator (select 4, 8, or 12 custom chocolate pieces)
`CHOCOLATES/src/context/CartContext.jsx`	Cart state management with custom gift note input field persistence
`CHOCOLATES/src/styles/App.css`	Warm chocolate amber palette styling, custom box visualizer, and checkout draw styles

3. Technical Features & Custom Box Builder

- **Custom Box Builder Component:** Allows users to pick box size (Small: 4 pieces, Large: 12 pieces) and select individual truffle flavors.
- **Gift Message Attachment:** Textarea input attached directly to the order object for customized gift deliveries.
- **Cocoa Content Filtering:** Filter truffles by cocoa intensity (Milk, 50% Dark, 70% Dark, 90% Extra Dark).

4. Code Highlights — Custom Gift Box Builder Logic (`BoxBuilder.jsx`)

```
// CHOCOLATES/src/components/BoxBuilder.jsx - Custom Gift Box Selection Logic
const addTruffleToBox = (truffle) => {
  if (selectedPieces.length >= maxBoxCapacity) {
    alert(`Box is full! Maximum capacity is ${maxBoxCapacity} pieces.`);
    return;
  }
  setSelectedPieces(prev => [...prev, truffle]);
};

const calculateBoxPrice = () => {
  const baseBoxFee = maxBoxCapacity === 12 ? 8.00 : 4.00;
  const piecesCost = selectedPieces.reduce((sum, item) => sum + item.price, 0);
  return baseBoxFee + piecesCost;
};
```

Submodule 4: COSTUMES Store & Rentals (`/COSTUMES`)

1. Module Overview & Business Model

The **COSTUMES Store Module** is a specialized retail and rental portal catering to theater productions, Halloween parties, cosplay conventions, and festival events. It provides both direct purchasing and daily rental options complete with deposit calculation and size measurement guides.

2. Architecture & File Breakdown

File Path	Component Role & Description
`COSTUMES/src/App.jsx`	Monolithic state engine (17.2 KB) managing rental date pickers, size selectors, and cart drawer
`COSTUMES/src/App.css`	Comprehensive stylesheet (16.6 KB) with theatrical dark theme tokens, size charts, and rental modals
`COSTUMES/src/main.jsx`	Root entry mounting React DOM tree to `<#root>` element

3. Technical Features & Rental Duration Calculation

- **Dual Transaction Model:** Toggle between 'Buy Outright' and 'Rent per Day' for each costume item.
- **Rental Date Range Estimator:** Calculates total rental duration in days from start/end date inputs and automatically adds a 50% refundable security deposit.
- **Interactive Size Matrix:** Size selector pills (XS, S, M, L, XL, Custom) displaying exact chest and waist measurement guidelines on hover.

4. Code Highlights — Rental Fee & Deposit Calculator (`App.jsx`)

```
// COSTUMES/src/App.jsx - Rental Fee & Security Deposit Estimator
const computeRentalQuote = (dailyRate, startIsoDate, endIsoDate) => {
  const start = new Date(startIsoDate);
  const end = new Date(endIsoDate);
  const diffMilliseconds = Math.max(0, end - start);
  const totalDays = Math.max(1, Math.ceil(diffMilliseconds / (1000 * 60 * 60 * 24)));
  const rentalTotal = dailyRate * totalDays;
  const refundableDeposit = dailyRate * 0.5;
  const grandTotal = rentalTotal + refundableDeposit;
  return { totalDays, rentalTotal, refundableDeposit, grandTotal };
};
```


Submodule 5: FRAGRANCE Depot (`/FRAGRANCE`)

1. Module Overview & Luxury Catalog Domain

The **FRAGRANCE Depot Module** is a luxury cosmetics storefront showcasing designer perfumes, eau de parfum, colognes, luxury scented candles, and aromatherapy essential oils. It features scent family categorization and dynamic volume bottle pricing.

2. Component Hierarchy & File Structure

File Path	Component Role & Description
`FRAGRANCE/src/App.jsx`	Main application engine (8.3 KB) managing scent category state, modal displays, and cart items
`FRAGRANCE/src/components/Header.js x`	Header bar featuring luxury brand logo, scent notes filter pills, and cart counter
`FRAGRANCE/src/components/ProductLi st.jsx`	Catalog list displaying fragrance bottles, scent notes tags, volume selectors, and Add button
`FRAGRANCE/src/App.css`	Luxury rose-gold and deep charcoal stylesheet (5.9 KB) styling bottle cards and cart drawer

3. Technical Features & Scent Profiling

- **Scent Profile Tags:** Categorizes items by scent notes: Floral, Woody, Oriental, Citrus, Fresh, and Aquatic.
- **Volume Selector Engine:** Interactive size pills (30ml, 50ml, 100ml) adjusting price dynamically per volume.
- **Fragrance Concentration Indicators:** Badges indicating Concentration levels (Eau de Toilette vs Eau de Parfum vs Pure Parfum Extract).

4. Code Highlights — Bottle Size Volume & Price Adjuster (`App.jsx`)

```
// FRAGRANCE/src/App.jsx - Bottle Size Price Multiplier Logic
const getAdjustedPrice = (basePrice, volumeMl) => {
  switch (volumeMl) {
    case 30: return basePrice;
    case 50: return Math.round(basePrice * 1.45);
    case 100: return Math.round(basePrice * 2.20);
    default: return basePrice;
  }
};
```

Submodule 6: BiteCourt FOOD Ordering (`/Food/Food`)

1. Module Purpose & Architecture

The **BiteCourt FOOD Ordering Module** is a food court ordering application offering fast food items, burgers, french fries, beverages, combo platters, and quick checkout. Built with a modular React architecture, it uses custom hooks and service layers.

2. Modular Layer Breakdown & Directory Blueprint

Directory Path	Architectural Layer & Description
`Food/Food/src/App.jsx`	Main router & layout wrapper integrating header, sidebar menu, and cart drawer
`Food/Food/src/services/api.js`	API service abstraction handling food menu fetching and order posting
`Food/Food/src/hooks/useCart.js`	Custom React hook encapsulating food cart context actions and calculations
`Food/Food/src/context/FoodContext.js` x`	Context provider managing cart items, meal add-ons, and promo discounts
`Food/Food/src/layouts/`	Layout components structuring food grid, sidebar categories, and checkout bar
`Food/Food/src/assets/`	High-quality food imagery asset folder (`french-fries.jpeg`)

3. Technical Features & Meal Customization

- **Combo Add-On Configurator:** Allows users to customize meals (Extra Cheese, Spicy Sauce, Large Drink).
- **Promo Code Validator:** Validates discount coupon codes (e.g. `BITENOW10` for 10% off).
- **Delivery Tip Selector:** Options for driver tip selection (\$2, \$3, \$5, Custom).

4. Code Highlights — Custom Hook Pattern (`useCart.js`)

```
// Food/Food/src/hooks/useCart.js - Custom Hook pattern for cart management
import { useContext } from 'react';
import { FoodContext } from '../context/FoodContext';
export const useCart = () => {
  const context = useContext(FoodContext);
  if (!context) {
    throw new Error('useCart must be executed within a FoodContextProvider wrapper');
  }
  return context;
};
```

Submodule 7: ICECREAMS Parlour Full-Stack (`/IceCreams`)

1. Full-Stack System Architecture Overview

The **ICECREAMS Parlour Module** is a full-stack web application featuring an Express REST API backend (`/IceCreams/server`) and a React client frontend (`/IceCreams/client`). It provides custom sundae building, topping selections, and order status tracking.

2. Client-Server Directory & API Endpoint Blueprint

Path / Endpoint	Role & API Specification
`IceCreams/client`	Vite + React frontend UI for custom ice cream scoop & topping customization
`IceCreams/server`	Express Node server exposing REST API endpoints for ice cream menu & order persistence
`GET /api/flavors`	Returns JSON array of available ice cream flavors (Vanilla, Chocolate, Mint, Strawberry, Mango)
`GET /api/toppings`	Returns JSON array of toppings (Choco Chips, Rainbow Sprinkles, Hot Fudge, Nuts)
`POST /api/orders`	Accepts order payload, stores order in memory database, returns created order with status 'Preparing'

3. Technical Features & Custom Sundae Builder

- **Interactive Sundae Configurator:** Select scoop count (1, 2, 3 scoops), container type (Waffle Cone, Sugar Cone, Tub), and multiple topping drizzles.
- **Order Lifecycle Tracker:** Status badge transitioning dynamically: Received -> Preparing -> Ready for Pickup.

4. Code Highlights — Express REST API Order Controller (`server/index.js`)

```
// IceCreams/server/index.js - Express API Controller
const express = require('express');
const app = express();
app.use(express.json());
const orders = [];

app.post('/api/orders', (req, res) => {
  const { customerName, flavor, scoops, container, toppings, totalAmount } = req.body;
  if (!customerName || !flavor) return res.status(400).json({ error: 'Missing order fields' });

  const newOrder = {
    id: `ICE-${Date.now()}`,
    customerName,
    flavor,
    scoops,
    container,
    toppings,
    totalAmount,
    status: 'Preparing'
  };
  orders.push(newOrder);
  res.status(201).json({ success: true, order: newOrder });
});
```

Submodule 8: SPORTS Goods & Fitness (`/SPORTS`)

1. Module Purpose & Sports Domain

The ****SPORTS Goods Module**** is an athletic equipment, activewear, and workout gear catalog. It covers gear for football, basketball, cricket, tennis, fitness gym equipment, protective pads, and workout accessories.

2. File Blueprint & Component Structure

File Path	Component Role & Description
`SPORTS/src/App.jsx`	Main application engine (11.7 KB) containing interactive grid, search bar, and equipment specification modals
`SPORTS/src/components/SportsCard.jsx`	Product card rendering equipment photo, sport category badge, stock status, and Quick Spec trigger
`SPORTS/src/components/FilterBar.jsx`	Filter bar with search input, sport category tabs, and in-stock toggle checkbox
`SPORTS/src/App.css`	Athletic emerald green stylesheet (2.2 KB) styling category tabs and gear specification modals

3. Technical Features & Spec Modal Views

- **Multi-Sport Filtering Tabs:** Filter items by sport discipline: Soccer, Cricket, Tennis, Basketball, Gym & Fitness.
- **Equipment Specifications Modal:** Modal view displaying weight, material composition (e.g. Carbon Fiber, Synthetic Leather), and warranty details.
- **In-Stock Filter Toggle:** Instant checkbox filter hiding out-of-stock equipment items.

4. Code Highlights — Product Specification Modal State Logic (`App.jsx`)

```
// SPORTS/src/App.jsx - Equipment Specification Modal Handler
const [selectedItem, setSelectedItem] =
  useState(null);
const openSpecModal = (item) => setSelectedItem(item);
const closeSpecModal = () =>
  setSelectedItem(null);
{selectedItem !== null && ( e.stopPropagation())> ■ {selectedItem.name}
Specifications
Material: {selectedItem.material} | Weight: {selectedItem.weight}
Warranty: {selectedItem.warranty}
Close Specifications
}}
```

Submodule 9: SHOES Footwear Emporium (`/Shoes`)

1. Module Overview & Product Domain

The ****SHOES Emporium Module**** is a sneaker and footwear catalog featuring athletic running shoes, formal leather shoes, basketball high-tops, casual sneakers, and sandals. It includes brand taxonomy filters and an interactive UK/US size selector.

2. File Blueprint & Component Architecture

File Path	Component Role & Description
`Shoes/src/Shoes.jsx`	Monolithic footwear application engine (20.0 KB) managing shoe brand filters, size pickers, and cart state
`Shoes/src/Shoes.css`	Footwear styling sheet (6.5 KB) with dynamic brand tag badges, size grid pills, and card hover effects
`Shoes/src/main.jsx`	Root mounting entry point connecting React component tree to DOM root

3. Technical Features & Size Matrix

- **Interactive UK/US Size Matrix:** Size selection pills (UK 6 to UK 11 / US 7 to US 12) with real-time stock indicator updates.
- **Brand Taxonomy Filtering:** Filter catalog by major footwear brands (Nike, Adidas, Puma, Jordan, Reebok).
- **Footwear Cushioning Specs:** Displaying midsole technology tags (Air Cushioning, Boost Foam, Gel Comfort).

4. Code Highlights — Shoe Size Matrix & Stock Availability (`Shoes.jsx`)

```
// Shoes/src/Shoes.jsx - Shoe Size Matrix Selector
const [selectedSize, setSelectedSize] = useState(null);
const sizes = [ { uk: 'UK 6', us: 'US 7', inStock: true }, { uk: 'UK 7', us: 'US 8', inStock: true }, { uk: 'UK 8', us: 'US 9', inStock: false }, { uk: 'UK 9', us: 'US 10', inStock: true }, { uk: 'UK 10', us: 'US 11', inStock: true } ];
const handleSizeSelect = (sizeObj) => { if (!sizeObj.inStock) return alert('Selected size is currently out of stock!'); setSelectedSize(sizeObj.uk); };
```

Submodule 10: TICKET BOOKING System (`/TICKETBOOKING`)

1. Module Overview & Entertainment Scope

The **TICKET BOOKING System Module** is a cinema and event ticket reservation platform. It includes an interactive 2D seating matrix, showtime scheduler, QR code generator, and a dedicated **Spiderman Special Feature Showcase** displaying movie schedules.

2. Component Hierarchy & Core Engine

File Path	Component Role & Description
`TICKETBOOKING/src/App.jsx`	Largest application engine in workspace (42.7 KB) managing cinema seat matrix, showtime schedules, and booking state
`TICKETBOOKING/src/TicketBooking.jsx`	Custom Spiderman movie schedule display component (1.2 KB) showing IMAX showtimes
`TICKETBOOKING/src/App.css`	Cinema theme styling sheet (3.1 KB) rendering dark theater seating charts, screen glow, and ticket pass modals

3. Technical Features & Digital QR Pass

- **2D Seating Matrix Grid:** Visual seat layout (Rows A-H, Seats 1-12) with occupied, available, and selected states.
- **Spiderman Movie Showcase:** Custom component highlighting Spiderman franchise showtimes, theater screen formats (IMAX 3D vs 2D), and cast details.
- **Digital QR Pass Generation:** Uses `react-qr-code` 2.2 to render a digital boarding ticket containing encrypted booking IDs upon confirmation.

4. Code Highlights — Dynamic Digital QR Pass Generation (`App.jsx`)

```
// TICKETBOOKING/src/App.jsx - Digital Pass QR Generation
import QRCode from 'react-qr-code';
const DigitalTicketModal = ({ ticketData, onClose }) => (
  <div>
    <div>Boarding Pass – {ticketData.movieTitle}</div>
    <div>Seats: {ticketData.selectedSeats.join(', ')} | Theater: Screen 3 (IMAX) Done</div>
  </div>
);
```

Technical Deep Dive & Cross-Cutting Architecture

1. Global State Management & Persistence Strategy

Across the project, state management balances high-performance local component state with global Context API providers. In the central hub (`src/App.jsx`), dark mode preferences are persisted across browser reloads using `localStorage` synchronization:

```
// src/App.jsx - LocalStorage Theme Sync Engine
const [darkMode, setDarkMode] = useState(() => {
  const saved = localStorage.getItem('darkMode');
  return saved !== null ? saved === 'true' :
    window.matchMedia('(prefers-color-scheme: dark)').matches;
});
useEffect(() => {
  document.documentElement.classList.toggle('dark-mode', darkMode);
  localStorage.setItem('darkMode', darkMode);
}, [darkMode]);
```

2. Design Tokens & Styling Architecture

The project strictly avoids heavy utility framework dependencies like TailwindCSS in favor of **Vanilla CSS Design Tokens** (`src/index.css` and `src/App.css`), ensuring custom glassmorphism, responsive grid layouts, and hardware-accelerated transitions.

CSS Variable Token	Default Light Mode Value	Dark Mode Override Value
--bg-primary	#F8FAFC (Ice Light)	#0F172A (Deep Slate Navy)
--text-primary	#1E293B (Dark Slate)	#F8FAFC (Pure Light)
--card-bg	rgba(255, 255, 255, 0.8)	rgba(30, 41, 59, 0.7)
--border-glow	rgba(59, 130, 246, 0.2)	rgba(96, 165, 250, 0.4)

3. Complete Repository Dependency Ecosystem

Package Name	Version	Architectural Role & Function in Workspace
react / react-dom	v19.2.7	Core UI component library & virtual DOM renderer engine
vite	v8.1.1	Next-generation ES Module bundler & rapid dev server
framer-motion	v12.43.0	Production motion engine powering carousel & card animations
axios	v1.18.1	Promise-based HTTP client for API requests
react-icons	v5.7.0	Icon packs (FontAwesome, Feather, Material Design Icons)
react-qr-code	v2.2.0	SVG QR code generator for digital ticket passes
react-router-dom	v7.18.2	Declarative SPA routing for submodule navigation

Developer Operations & Strategic Roadmap

1. Developer Setup & Execution Blueprint

To launch the central hub or any sub-application independently, execute the following commands:

```
# Command Sequence A: Launch Central Nithya Mart Portal Hub cd c:\Nithin_Academic\MERN_Stack_Team_4 npm install npm run dev # Portal active at: http://localhost:5173
```

```
# Command Sequence B: Launch Submodule Independently (e.g. BAG or TICKETBOOKING) cd BAG # or cd TICKETBOOKING, cd CHOCOLATES, etc. npm install npm run dev
```

2. Git Subtree Operational Workflows

For ongoing maintenance and team updates, use subtree pull/push commands:

```
# Pull latest updates from developer branch into prefix folder git subtree pull --prefix=BAG origin feature/bag-store --squash # Push localized fixes back to submodule remote branch git subtree push --prefix=TICKETBOOKING origin feature/ticket-booking
```

3. Future Enhancements & Strategic Roadmap

- **Unified JWT Authentication Microservice:** Centralize user authentication across all 10 sub-modules using JWT SSO tokens.
- **Cross-Module Multi-Merchant Cart:** Enable users to combine products from BAG, CHOCOLATES, and SHOES into a single checkout flow.
- **Automated CI/CD Pipeline:** Deploy GitHub Actions to automatically run unit tests and deploy static builds to AWS CloudFront / Vercel.

End of Master Architectural Documentation — MERN Stack Team 4 (Nithya Mart)